ATM Sentinel

On-device machine learning for ATM health, tamper and cash monitoring

Presentation notes · IoT semester project · October 2026

How to use these notes

- Sections 1–3 give you the story: the opening pitch, the problem and the architecture. Learn the pitch by heart.
- Sections 4–9 are the technical part, in the order you would explain them: hardware, data, the health model, the audio module, firmware and the web side. Each ends with a short *Say* box.
- Section 10 is the live demo script with a checklist. Section 11 lists the honest limitations, and section 12 has likely questions with answers.
- Every number here comes from the project's own result files (`ml/evaluation_metrics.json` and the model reports). None of the accuracy figures were measured on the ESP32 itself.

Contents

1. The 60-second pitch	2. Problem and goal
3. System architecture	4. Hardware
5. Data and the simulator	6. Health model (two-stage cascade)
7. Audio TinyML and visit fusion	8. Firmware
9. Backend and dashboards	10. Live demo script
11. Limitations, said honestly	12. Likely questions

1. The 60-second pitch

> **Say:** *ATMs fail in boring ways: the cooling fan dies, the power sags, a sensor drifts, someone tampers with the machine, or it stops giving out cash. Banks usually find out when a customer complains. ATM Sentinel is a small ESP32 node that sits inside the ATM, samples six kinds of sensors every second, and runs two neural networks* **on the chip itself** *to decide whether the machine is healthy and, if not, what kind of fault it is. A third tiny model listens to the dispenser sound. The node sends only its readings and decisions over WiFi and MQTT, never raw audio. Staff get an operations dashboard, and customers get a public map that tells them which ATMs are working and when cash was last withdrawn.*

Three things make the project more than a sensor logger. Use them as your headline points:

- **The ML runs on the microcontroller.** Both health models are INT8 TensorFlow Lite networks with about 1,100 parameters each, using roughly 1 KB of working memory each. The node decides locally and does not depend on a server.
- **The training data is physically modelled.** Faults are injected into a thermal and electrical model of a cabinet, not painted onto a channel, so the patterns across channels are realistic (for example, a failing fan draws *more* current before the temperature rises).
- **It is honest about its numbers.** Test data comes from ATM units the model never saw, and every weakness is measured and reported (sensor-fault recall, the size of the audio dataset).

2. Problem and goal

- **Problem:** ATM faults are found late. Overheating, power trouble and tampering often show in sensor data minutes to hours before the machine fails or a customer notices.
- **Goal:** a low-cost node that (1) detects and classifies faults in real time, (2) raises tamper alerts quickly, (3) reports cash-dispense activity, and (4) shows customers a simple Working / Out of service map.
- **Constraint:** it must run on a $5-class microcontroller, so models must be tiny, integer-quantised and fast.

Fault class	What it means physically	Main signals
COOLING_FAILURE	Fan degrading or stalled; cabinet heats up	Temperature rises as a decelerating curve, fan-rail current changes
HIGH_HUMIDITY	Moisture getting into the cabinet	Humidity and absolute humidity both rise
POWER_FAILURE	Mains sag, swell or outage	AC voltage; in an outage the fan stops and the cabinet heats
TAMPER	Someone forcing or shaking the machine	Bursts of SW-420 vibration, motion
SENSOR_FAULT	A sensor stuck, biased, drifting or missing	Missing readings, frozen values, implausible relationships between channels

3. System architecture

On the ATM: ESP32 node	Link	Laptop	People
Sensors: BME280 (temperature, pressure), VEML7700 (light), SW-420 (vibration), HC-SR501 (motion), INMP441 (sound); voltage and current simulated 1 Hz sampling into a 300 s window Every 60 s: 57 features → Stage 1 → Stage 2 (INT8 TFLite) Audio: 36 features/s → INT8 MLP; + vibration + PIR → one event per visit	WiFi → MQTT one topic set per ATM	Mosquitto broker → logger (mqtt_to_sqlite.py) → SQLite file → Flask web app	Customers: public map (/) Staff: operations console (/admin) Demo: phone controls (/demo)

MQTT topic	Sent	Contents
atm/ATM_001/telemetry	every 1 s	the 10-field canonical record (temperature, pressure, light, voltage, current, vibration, motion…)
atm/ATM_001/inference	every 60 s	Stage-1 probability, Stage-2 probabilities, final class, persistence, alert, top features, all 57 raw features
atm/ATM_001/event	per visit, plus an idle summary each minute	visit result: cash dispensed / not dispensed / tamper…, with the audio, vibration and presence figures
atm/ATM_001/status	at boot (retained)	IP, which sensors and models started, configuration
atm/ATM_001/cmd	on demand	commands to the node (used by the demo controls)

Why MQTT: lightweight publish/subscribe, made for unreliable links and small devices; the broker decouples the node from whoever consumes the data (logger, dashboard, future services). **Why SQLite:** a single file, no server, enough for a fleet this size. The original design had an ESP8266 gateway; the node now connects to WiFi directly, so the gateway is not built.

> Say: *Everything that needs intelligence happens on the node. The laptop only stores and displays. If the server goes down, the ATM node still knows its own state.*

4. Hardware

Board: **DOIT ESP32 DevKit V1** (dual-core 240 MHz, 520 KB SRAM, 4 MB flash, WiFi). All I2C sensors share one bus at 100 kHz.

Part	Measures	Interface / pin	Status in the demo
BME280	temperature, pressure, (humidity)	I2C 0x77 (SDA 21, SCL 22)	Temperature and pressure live. Humidity disabled: its sensing element stuck at 100 %.
VEML7700	ambient light (lux)	I2C 0x10	Live
SW-420	vibration: falling edges per second (5 ms debounce, max 200)	GPIO 26, interrupt	Live
HC-SR501 PIR	motion (latched ~3 s)	GPIO 27	Live (ignored for the first 60 s while it warms up)
INMP441	sound, 16 kHz, 24-bit	I2S: SCK 32, WS 25, SD 33	Live (connection was unreliable during testing)
INA219	12 V fan-rail current	I2C 0x40	Simulated (1.417 A ± noise): not detected on the bus

Part	Measures	Interface / pin	Status in the demo
ZMPT101B	AC mains voltage	GPIO 34 (reserved)	Simulated (230 V ± 0.4 V): not wired, for safety

- **SW-420 counts edges, not strength.** It is an on/off contact that chatters when shaken, so the firmware counts falling edges per second. That is why the data never claims to measure vibration amplitude.
- **Calibration knobs are named constants** in the firmware (pressure offset +37 hPa for this site, light scale, simulated voltage and current levels), all marked PROVISIONAL so nothing is hidden.
- **Power:** the PIR needs more than 4.5 V and is fed from VIN (~4.7 V over USB); a 470 µF capacitor on 3.3 V prevents brownouts.

> **Say:** *Two channels are simulated, and I say that up front: mains voltage is dangerous to wire on a breadboard, and the current sensor isn't working. The simulated values follow the same schema and noise levels, so the pipeline is identical. Swapping in the real sensors is a one-line change in the firmware.*

5. Data and the simulator

One canonical record (`schema.yaml`), 10 fields at 1 Hz, used identically by the simulator and the firmware. Three rules:

- **Missing is null**, never 0 or −999. Missing readings are themselves evidence of a sensor fault.
- **Labels never appear in telemetry.** Ground truth is stored separately, so the stream looks exactly like real hardware output.
- **Vibration is an edge count; motion is a latched yes/no.** The data describes what the sensors actually report.

Why simulate? You cannot wait for real ATMs to overheat or be attacked, and there is no public dataset of ATM cabinet telemetry. The simulator (`atmsim`) models the physics in six layers:

Layer	What it models
Environment	outdoor daily temperature and humidity cycle, atmospheric pressure tides
Lobby	air-conditioning switching on and off, daylight through glass
Cabinet thermal	a first-order heat model: a weaker fan both raises the final temperature and slows the approach to it
Psychrometrics	absolute humidity is the real quantity; relative humidity is derived from it (Magnus formula)
Events	customer footfall (Poisson), PIR latching, vibration bursts, heat from transactions
Sensor emulation	per-device calibration bias, datasheet noise, rounding, range limits, dropouts

Faults are injected into the *physics*, so their effects across channels are consistent. Example: when a cabinet heats at constant moisture, relative humidity *falls*. Only real moisture ingress raises absolute humidity, which is how the model can tell cooling failure from a humidity fault.

Split	Windows	How it is separated
Train	6,900 (1,150 per class, 60 s stride)	31 virtual ATM units
Validation	790 (300 s stride, no overlap)	7 different units
Test	836 (300 s stride, no overlap)	6 different units, never seen in training

- **Features:** 57 per 300-second window: per-channel mean, std, min, max, slope, change and missing fraction; voltage sag/swell/dropout fractions; vibration and motion statistics; absolute humidity, dew point and the temperature–humidity correlation.
- **Scaling:** mean and std are fitted on the training split only, then baked into the firmware as constants.
- **Why 300 s windows:** thermal faults move slowly. Over 30 s, a degrading fan changes temperature by about 0.05 °C, which is inside sensor noise.

> **Say:** *We split by ATM unit, not by random rows. Each simulated device has its own calibration bias, so a random split would let the model memorise devices. Testing on unseen units is what a real fleet rollout looks like.*

6. Health model: the two-stage cascade

	Stage 1: is it healthy?	Stage 2: which fault?
Network	MLP 57 → 16 → 8 → 1 (sigmoid)	MLP 57 → 16 → 8 → 5 (softmax)
Parameters	1,073	1,109
Runs	every 60 s on the last 300 s	only when Stage 1 says abnormal
Decision	abnormal if p ≥ 0.7615 (chosen on validation only, maximising balanced accuracy)	highest-probability fault class
On the ESP32	INT8 TFLite Micro, ~1 KB working memory	INT8 TFLite Micro, ~1.1 KB working memory

- **Why two stages:** most of the time the ATM is healthy. Stage 1 is a cheap filter; Stage 2 only runs when needed and only has to learn the differences between faults. Each stage can be tuned and evaluated separately.
- **Sensor checks before the model:** if a channel is missing for 60 s, or every reading is identical (stuck), the node reports SENSOR_FAULT straight away without asking the models.
- **Persistence:** an ALERT is raised only after **2 consecutive** abnormal windows, which removes one-off false alarms. The customer map shows *Out of service* only on an alert (a sensor fault alone doesn't make the ATM unusable).
- **Explainability:** each inference reports the 5 features furthest from normal (largest |z|), shown on the admin page.

Results on the held-out test split (836 windows)

Model	Accuracy	Other key numbers
Stage 1 (healthy vs fault)	0.886	ROC-AUC 0.910 · fault recall 0.853 · fault precision 0.984 · normal recall 0.967
Stage 2 (fault type, given a fault)	0.885	macro-F1 0.883 · balanced accuracy 0.892
Full cascade, 6 classes	0.827	macro-F1 0.804 · balanced accuracy 0.815
Full cascade, humidity disabled (as deployed)	0.693	same as above for every class except HIGH_HUMIDITY, which drops to 0

Class (full cascade)	Recall	Precision	Comment
NORMAL	0.967	0.732	few false alarms; some faults slip through as normal
COOLING_FAILURE	0.955	0.944	
HIGH_HUMIDITY	0.938	0.839	0 with the humidity sensor disabled
POWER_FAILURE	0.802	0.802	19 short sags look normal to Stage 1
TAMPER	0.993	0.993	the strongest class
SENSOR_FAULT	0.234	0.762	weakest: 60 of 137 windows pass Stage 1 as normal (subtle bias/drift)

Say: *Stage 1 sits at 0.89, not 0.99, and that is intended. A synthetic dataset that scores 0.99 would mean the faults were injected too cleanly to be worth modelling. Our weakest class is sensor fault: slow drift and bias look almost normal for 5 minutes. The deterministic checks catch the obvious cases (missing or stuck sensors) before the model is asked.*

7. Audio TinyML and visit fusion

A second module runs **beside** the cascade and never changes it. It answers: did a customer visit end with cash being dispensed?

Step	What happens
Capture	INMP441 microphone over I2S at 16 kHz, read in 16 ms blocks by a task pinned to CPU core 0
Features	every ~1 s: 36 features (13 MFCCs, loudness, spectral centroid, roll-off, flatness, ...) computed on the chip. No audio is stored
Classifier	INT8 MLP 36 → 16 → 3: BACKGROUND / DISPENSE / ABNORMAL (3,144-byte model)
Loudness gate	a window quieter than −50 dBFS counts as background, whatever the model says (site-calibrated constant)
Fusion	per visit: audio summary + SW-420 vibration + PIR presence + the cascade's tamper flag go to a 31-node decision tree with safety rules
Output	one event per visit on atm/<id>/event: Cash dispensed, Not dispensed (abnormal mechanism), Suspected tamper, ...

- **Safety rules:** the cascade's tamper decision always wins; tamper-level vibration can never count as cash dispensed; dispense sound with nobody present is not counted; low-confidence visits are marked unclear instead of guessed.
- **Privacy:** only the event summary leaves the node, never raw audio.
- **Cost on the ESP32** (measured on the board): about 3.9 % of one CPU core, 0.3 ms per classification, free heap steady at ~145 KB, 0 I2S read errors. Firmware grew by 47.9 KB flash and 19.3 KB RAM.

Results (PC, held-out data)

Check	Result
Audio classifier, test	accuracy 0.970 but **macro-F1 0.726** (the honest number; easy synthetic background windows inflate accuracy)
Leave-one-recording-out	mean accuracy 0.81 across held-out recordings
Float vs INT8 model	100 % agreement
Fusion tree alone / with safety rules	0.892 / 0.853 accuracy; tamper recall 1.000
Firmware vs Python reference	36/36 identical decisions; max feature difference 7.9×10^{-6}

> **Say:** *The audio part is a proof of the pipeline more than a finished detector. It was trained on about 37 seconds of internet recordings, so it confuses loud speech with the dispenser. The fix is known: record a few minutes of real background and dispenser sound through this microphone and retrain. The training script and the recorder for that are already in the project.*

8. Firmware

Part	Details
Main loop (core 1)	samples all sensors once per second into a 300-sample ring buffer and publishes telemetry
Inference	every 60 s once the window is full (first result 5 min after boot): 57 features → scaling → Stage 1 → Stage 2
Audio task (core 0)	reads the microphone continuously so it never delays the 1 Hz sampler
Time	NTP (pool.ntp.org with Google and Cloudflare fallbacks); until synced, timestamps are seconds since boot
Network	WiFi station + MQTT (PubSubClient, 2 KB buffer), reconnecting every 5 s if lost
Parity tests	the firmware's feature code is checked against the Python simulator's features (test_features.py); the audio code against the Python reference (host build of the same C++ files)
Humidity	ENABLE_HUMIDITY 0: the 12 humidity-derived model inputs are fed the training average (z = 0) and humidity is left out of telemetry. Measured effect: only HIGH_HUMIDITY detection is lost

> **Say:** *The firmware and the training code share one contract: the same 57 features in the same order with the same scaling. We test that contract on the PC, so the model on the chip sees exactly what it was trained on.*

9. Backend and dashboards

- **Logger** (`tools/mqtt_to_sqlite.py`): subscribes to atm/+/#, writes four tables: telemetry, inference, status, event.
- **Web app** (`tools/dashboard.py`, Flask): reads SQLite, serves three pages and a small JSON API.

Page	Who	What it shows
/ (map)	public, no login	Leaflet + OpenStreetMap of Coimbatore. Each ATM: Working / Out of service / Status unknown (no data for 30 s), last report, last cash withdrawal. A failed withdrawal turns the ATM to *Out of service · last cash withdrawal failed* until a later withdrawal succeeds. Customers never see fault types or sensor data
/admin	staff login	fleet list with fault class and alert state; live charts of each sensor (last 300 s); Stage-1 probability vs threshold; Stage-2 probabilities; top features; inference history; *Activity*: cash dispensed count, last dispense, and a table of Cash dispensed / Not dispensed events
/demo	demo login	two large buttons, *Cash dispensed* and *Not dispensed*, for driving the demo from a phone

- **Two modes** on both pages: *Live + simulated* (Amrita plus 9 simulated ATMs around Coimbatore, to show a fleet) and *Live only* (just the real node). Simulated ATMs are labelled as such.
- **Security:** passwords come from environment variables, comparisons are constant-time, cookies are SameSite=Lax, and the public API returns only location and a yes/no status.

About the demo buttons (be ready to explain)

In a real deployment the ATM's own controller knows when it dispensed cash. The buttons stand in for that signal: they send `txn:1` or `txn:0` to the node over MQTT. The node closes the current visit with that outcome, attaches the presence, vibration and sound it actually measured during the visit, and stores it with the rule `CONFIRMED` (the audio model's own decisions are stored as `MODEL`). Tamper still overrides a button press. If asked, say exactly this; it is also how real monitoring cross-checks bank transactions against what the sensors heard.

10. Live demo script

Before the evaluation (15 minutes)

```
# 1. broker (config: listener 1883 0.0.0.0 / allow_anonymous true)
mosquitto -c /tmp/mosq.conf -v
# 2. logger
.venv/bin/python tools/mqtt_to_sqlite.py
# 3. clear old data, then start the web app
sqlite3 atm_sentinel.db "DELETE FROM telemetry; DELETE FROM inference; DELETE FROM status;
DELETE FROM event; VACUUM;"
ATM_ADMIN_PASSWORD=... ATM_DEMO_PASSWORD=... ATM_SECRET_KEY=... .venv/bin/python
tools/dashboard.py
```

- Power the ESP32 **at least 6 minutes early**: the first health result needs a full 5-minute window.
- If the network changed, the laptop's IP changes: update `MQTT_HOST` in the firmware and reflash.
- Serial Monitor: telemetry lines once a second, `MQTT connected`, and later `INFER … class=NORMAL`.
- Point the PIR at where the 'customer' will stand, not at the audience. Open `/admin` on the laptop (*Live only*) and the map in another tab; sign in to `/demo` on the phone.

During the evaluation (about 6 minutes)

#	Do	Show / say
1	Open the map	Amrita is *Working*; status refreshes every 5 s; customers see only yes/no. Switch to *Live + simulated* to show a fleet with faults.
2	Open the admin page	Live sensor charts (1 Hz), Stage-1 probability under the threshold, class NORMAL, top features.
3	Withdrawal	Customer stands at the ATM, dispense sound plays, press *Cash dispensed*: a row appears in Activity, the count goes up, and the map shows *Last cash withdrawal: just now*.
4	Failed withdrawal	Play the 'not working' sound, press *Not dispensed*: a red row in Activity; the map turns Amrita *Out of service · last cash withdrawal failed*. Then do step 3 again to bring it back to Working.
5	Tamper	Shake the SW-420 hard for ~20 s. Within a minute or two the cascade reports TAMPER; after 2 windows it raises an ALERT and the map shows Out of service. It stays for ~5 minutes on purpose, because the 300 s window still contains the shaking.
6	Close	Point to the evaluation numbers (section 6) and the limitations (section 11).

If something fails: no WiFi → show the Serial Monitor (inference runs on the chip anyway); no node data → switch both pages to *Live + simulated*, which shows the full system with simulated units.

11. Limitations, said honestly

Limitation	Why it matters	What would fix it
Trained on simulated data	Accuracy is measured on the simulator, not on real ATMs	Calibrate and fine-tune with logged data from real cabinets
Voltage and current simulated	POWER_FAILURE and fan-current evidence cannot come from this node yet	Wire the ZMPT101B safely and fix the INA219 connection
Humidity sensor failed	HIGH_HUMIDITY can't be detected (measured: only that class is lost)	Replace the BME280, set ENABLE_HUMIDITY 1
Sensor-fault recall 0.23	Slow drift and bias look normal over 5 minutes	Longer windows or a dedicated drift detector
Audio data is tiny	~37 s of internet clips; loud speech can look like dispensing	Record real dispenser and lobby audio through the INMP441 and retrain
Tamper threshold provisional	75 edges/s comes from synthetic data	Measure real SW-420 counts on an enclosure
Tamper status persists ~5 min	The 300 s window still contains the event	Intended for safety; a separate fast tamper path could clear sooner
No buffering when offline	Readings during a WiFi drop are not sent later	Store-and-forward queue on the node
Single node, no TLS	Prototype security on a local network	MQTT over TLS with per-device credentials

Future work

- Real sensor data from an ATM enclosure; fine-tune the cascade on it.
- Proper audio dataset and retraining; reach a usable macro-F1 for dispense vs abnormal.
- Cross-check events against the bank's transaction log (cash dispensed per the bank, but no sound or notes detected → jam or fraud).
- Over-the-air firmware updates and a store-and-forward buffer.

12. Likely questions

Q. Why not just use thresholds?

Simple thresholds catch the obvious cases, and we use them (missing and stuck sensors, the tamper vibration level). But faults like a degrading fan show as a *combination*: slightly higher current, a slow temperature curve, a falling relative humidity. A small network learns those combinations; it is still only ~1,100 parameters.

Q. Why run the ML on the ESP32 instead of a server?

It works without the server or network, it sends decisions instead of raw data (less bandwidth, more privacy), and it reacts within one inference cycle. INT8 quantisation makes each model about 1 KB of working memory.

Q. How do you know the simulated data is realistic?

Every constant is justified in qa/CALIBRATION.md (datasheet noise, thermal time constants, mains tolerances), faults act on the physics, and a QA report checks the statistics. It is described as documented synthetic data generation, not as real ATM data.

Q. Isn't 0.83 accuracy low?

It's on unseen ATM units, with deliberately realistic noise. Stage 1 is 0.89 with 98 % precision on faults, and the 2-window persistence rule cuts false alarms further. The weak class (sensor fault) is reported, not hidden.

Q. What happens if WiFi drops?

The node keeps sampling and inferring, and reconnects every 5 s. Readings during the outage are not re-sent; the map marks the ATM 'Status unknown' after 30 s without data.

Q. Why does tamper stay on after shaking stops?

The model looks at the last 300 s, so the shaking stays in the window for about 5 minutes. For security that is the safe behaviour: an attacked ATM shouldn't look safe seconds later.

Q. How is the audio private?

Audio never leaves the chip. Only per-visit numbers (probabilities, durations) are sent.

Q. How did you check the firmware matches the Python models?

The same C++ files are compiled on the PC and compared with the Python reference: 36/36 audio decisions match, features within 8×10^{-6}; the cascade feature code is tested against the simulator's features.

Q. Where does 'Cash dispensed' come from in the demo?

From the confirmation signal sent by the demo controls (standing in for the ATM's transaction system), combined with the node's measurements of that visit. It is stored as CONFIRMED; the audio model's own detections are stored as MODEL.

Q. How would this scale to many ATMs?

Each node does its own inference and publishes small messages, so the server only stores and displays. Add nodes by giving each an ATM_ID and a location; MQTT topics are already per ATM (atm/<id>/...).